

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations. (GUI)

```
import tkinter as tk
from tkinter import messagebox
import random

def is_prime(n, k=20):
    if n < 2:
        return False
    for p in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]:
        if n % p == 0:
            return n == p
    d, r = n - 1, 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def gen_prime(bits):
    while True:
        n = random.getrandbits(bits) | (1 << bits - 1) | 1
        if is_prime(n):
            return n

def egcd(a, b):
    if a == 0:
        return b, 0, 1
    g, x, y = egcd(b % a, a)
    return g, y - (b // a) * x, x
```

```
def modinv(a, m):
    g, x, _ = egcd(a, m)
    return x % m

def generate_keys(bits=512):
    p = gen_prime(bits)
    q = gen_prime(bits)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    d = modinv(e, phi)
    return n, e, d

def encrypt(message, e, n):
    m = int.from_bytes(message.encode(), "big")
    if m >= n:
        raise ValueError("Message too large for key size")
    return pow(m, e, n)

def decrypt(cipher, d, n):
    m = pow(cipher, d, n)
    length = (m.bit_length() + 7) // 8
    return m.to_bytes(length, "big").decode()

def do_genkeys():
    n, e, d = generate_keys(int(bits_var.get()))
    n_var.set(str(n))
    e_var.set(str(e))
    d_var.set(str(d))

def do_encrypt():
    try:
        cipher = encrypt(msg_var.get(), int(e_var.get()),
int(n_var.get()))
        cipher_var.set(str(cipher))
    except Exception as ex:
        messagebox.showerror("Error", str(ex))

def do_decrypt():
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

```
        try:
            plain = decrypt(int(cipher_var.get()), int(d_var.get()),
int(n_var.get()))
            result_var.set(plain)
        except Exception as ex:
            messagebox.showerror("Error", str(ex))

root = tk.Tk()
root.title("RSA - GUI")

bits_var = tk.StringVar(value="512")
n_var = tk.StringVar()
e_var = tk.StringVar()
d_var = tk.StringVar()
msg_var = tk.StringVar()
cipher_var = tk.StringVar()
result_var = tk.StringVar()

tk.Label(root, text="Key size (bits)").grid(row=0, column=0, padx=5,
pady=5)
tk.Entry(root, textvariable=bits_var, width=10).grid(row=0, column=1,
sticky="w", padx=5, pady=5)
tk.Button(root, text="Generate Keys", command=do_genkeys).grid(row=0,
column=2, padx=5, pady=5)

tk.Label(root, text="n").grid(row=1, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=n_var, width=60, state="readonly").grid(row=1,
column=1, columnspan=2, padx=5, pady=5)

tk.Label(root, text="e").grid(row=2, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=e_var, width=60, state="readonly").grid(row=2,
column=1, columnspan=2, padx=5, pady=5)

tk.Label(root, text="d").grid(row=3, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=d_var, width=60, state="readonly").grid(row=3,
column=1, columnspan=2, padx=5, pady=5)

tk.Label(root, text="Message").grid(row=4, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=msg_var, width=40).grid(row=4, column=1,
padx=5, pady=5)
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

```
tk.Button(root, text="Encrypt", command=do_encrypt).grid(row=4, column=2,
padx=5, pady=5)

tk.Label(root, text="Cipher").grid(row=5, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=cipher_var, width=40).grid(row=5, column=1,
padx=5, pady=5)
tk.Button(root, text="Decrypt", command=do_decrypt).grid(row=5, column=2,
padx=5, pady=5)

tk.Label(root, text="Result").grid(row=6, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=result_var, width=40,
state="readonly").grid(row=6, column=1, padx=5, pady=5)

root.mainloop()
```

The screenshot shows a window titled "RSA - GUI" with a standard Windows title bar (minimize, maximize, close buttons). The window contains several input fields and buttons:

- Key size (bits):** A text box containing "512".
- Generate Keys:** A button to the right of the key size field.
- n:** A text box containing the large prime number: 135897402598650562841921348622120218170004695865492269154471.
- e:** A text box containing the public exponent: 65537.
- d:** A text box containing the private exponent: 132444861662587679476314148941698011427964965352712547346298.
- Message:** A text box containing "hiiiiiiii".
- Encrypt:** A button to the right of the message field.
- Cipher:** A text box containing the encrypted result: 2343286427004465052744354164440093113310.
- Decrypt:** A button to the right of the cipher field.
- Result:** A text box containing "hiiiiiiii".

The background of the window shows a dark theme with some code snippets visible, such as `tk.Label(root, text="n").grid(row=1, column=0, padx=5, pady=5)` and `tk.Label(root, text="Result").grid(row=6, column=0, padx=5, pady=5)`.

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations. (CLI)

```
import sys
import random

def is_prime(n, k=20):
    if n < 2:
        return False
    for p in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]:
        if n % p == 0:
            return n == p
    d, r = n - 1, 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def gen_prime(bits):
    while True:
        n = random.getrandbits(bits) | (1 << bits - 1) | 1
        if is_prime(n):
            return n

def egcd(a, b):
    if a == 0:
        return b, 0, 1
    g, x, y = egcd(b % a, a)
    return g, y - (b // a) * x, x
```

```
def modinv(a, m):
    g, x, _ = egcd(a, m)
    return x % m

def generate_keys(bits=512):
    p = gen_prime(bits)
    q = gen_prime(bits)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    d = modinv(e, phi)
    return n, e, d

def encrypt(message, e, n):
    m = int.from_bytes(message.encode(), "big")
    if m >= n:
        raise ValueError("Message too large for key size")
    return pow(m, e, n)

def decrypt(cipher, d, n):
    m = pow(cipher, d, n)
    length = (m.bit_length() + 7) // 8
    return m.to_bytes(length, "big").decode()

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python 3B_rsa_cli.py genkeys <bits>")
        print("      python 3B_rsa_cli.py encrypt <message> <e> <n>")
        print("      python 3B_rsa_cli.py decrypt <cipher> <d> <n>")
    else:
        mode = sys.argv[1]
        if mode == "genkeys":
            bits = int(sys.argv[2]) if len(sys.argv) > 2 else 512
            n, e, d = generate_keys(bits)
            print("n =", n)
            print("e =", e)
            print("d =", d)
        elif mode == "encrypt":
            message, e, n = sys.argv[2], int(sys.argv[3]),
            int(sys.argv[4])
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

```
        print(encrypt(message, e, n))
    elif mode == "decrypt":
        cipher, d, n = int(sys.argv[2]), int(sys.argv[3]),
int(sys.argv[4])
        print(decrypt(cipher, d, n))
    else:
        print("Invalid mode")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 2ndcli.py encrypt "hello" 65537
IndexError: list index out of range
• PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 2ndcli.py genkeys 512
n = 1672623815657950113215085161062132412205116047861906778766843636520922215917735040984169311306759204382610418605837703901395549222807290900
83177078974342187017024099078289154924756005605324499476840743102761315754424156692268028187036257162508046091175716373578082336270463655477141
466422399855830394976369811
e = 65537
d = 327445009000892624815745954444689083813973617051171700196622344105380476711558444211192496462413688790895474420998480408762210129944541595
99487605969270782544450983776542683496307336135953013595983650140969750167709164844787674102081644344485710540929412353180359330479384435000085
46549758045562530052267153
• PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 2ndcli.py encrypt "hello" 65537 n = 1672623815657950113215085161062132412
20511604786190677876684363652092221591773504098416931130675920438261041860583770390139554922280729090083177078974342187017024099078289154924756
005605324499476840743102761315754424156692268028187036257162508046091175716373578082336270463655477141466422399855830394976369811
```